

Lernziele:

- Vertiefung des Umgangs mit einfach verketteten Listen (vgl. Blatt 12)
- Verwendung des Schlüsselwortes `readonly`
- The Good, the Bad and the Ugly (im Kontext von Code, Design und überhaupt)

Vorbemerkung

Wir machen weiter mit dem, was wir in Session 12 zum Thema verkettete Listen programmiert haben. Ihr könnt dazu euren eigenen Code von Session 12 verwenden oder den Code von der Webseite (zu finden unter Session 16) laden. Hier das Programm in verkürzter Form:

```
1 @ <stdio.h>
2 @ <stdlib.h>
3
4 struct Node
5 {
6     val: int;
7     next: -> Node;
8 };
9
10 fn prependToList(list: -> -> Node, val: int)
11 { /* ... */ }
12
13 fn printList(list: -> Node)
14 { /* ... */ }
15
16 fn printListRec(list: -> Node)
17 { /* ... */ }
18
19 fn printListReverse(list: -> Node)
20 { /* ... */ }
21
22 fn freeList(list: -> Node)
23 { /* ... */ }
24
25 fn main()
26 {
27     local list: -> Node = nullptr;
28
29     for (local i: int = 1; i <= 10; ++i) {
30         prependToList(&list, i);
31     }
32
33     printf("list:\n");
34     printList(list);
35
36     printf("list:\n");
37     printListRec(list);
38
39     printf("list in reverse order:\n");
40     printListRec(list);
41
42     freeList(list);
43 }
```

Aufgabe

Macht euch zur Erinnerung nochmals mit dem Code und der Datenstruktur vertraut. Idealerweise so, dass ihr alles eurem Nebensitzer erklären könnt.

Schritt 2: Motivation für das Schlüsselwort `readonly`

Wenn eine Funktion zur Ausgabe einer Liste aufgerufen wird, erwartet man, dass dabei die Liste nicht verändert wird. Aber nehmen wir an, dass durch ein Copy-&-Paste-Missgeschick die Funktion wie folgt verändert wurde:

```
fn printList(list: -> Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val *= 2;
    }
}
```

Nachdem der Wert eines Listenknotens ausgegeben wurde, wird der Wert verdoppelt. Nehmt im Code die Änderung vor und testet das Programm. Dabei werdet ihr natürlich beobachten, dass sich die Liste jedes Mal verändert, wenn man `printList` verwendet.

Nun ändert die Funktion wie folgt ab:

```
fn printList(list: -> readonly Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val = 42;
    }
}
```

Der Parameter `list` ist nach wie vor ein Zeiger auf einen Listenknoten, neu ist jedoch, dass hier das Schlüsselwort `readonly` verwendet wird. Die Bedeutung ist in diesem Fall, dass in der Deklaration vereinbart wird, dass das, was am Ende des Zeigers liegt, nicht verändert werden darf. Übersetzt man nun das Programm, liefert der Compiler eine Fehlermeldung:

```
MCL:session16 lehn$ abc list_1.abc
list_1.abc:26.14-26.17: error: : assignment of read-only variable 'list->val'

        list->val = 42;
            ^^
list_1.abc:26.18-26.20: error: operator can not be applied to operand 'list->val' of type
'const int' and operand '42' of type 'i32'
```

Wieso ist es gut, dass wir einen Fehler bekommen? Natürlich sind Fehler schlecht, aber es ist besser, einen Fehler *beim Übersetzen* zu bekommen als *zur Laufzeit*. Mit dem Schlüsselwort `readonly` haben wir dokumentiert und dem Compiler mitgeteilt, dass diese Funktion den Inhalt eines Listenknotens (also das, was am Ende des Zeigers auf den Listenknoten liegt) nicht verändern darf. Der Compiler erkennt, dass dieses Versprechen in Zeile 26 gebrochen wird. Wir können uns jetzt dafür entscheiden, dass die Funktion doch Knoten ändern darf (indem man `readonly` entfernt) oder den Fehler (d.h. die unabsichtliche Veränderung des Knotens) beheben.

Schritt 3: Generelle Verwendung von `readonly`

Die Verwendung von `readonly` ist nicht auf Zeigertypen beschränkt, sondern kann für jeden Variablentyp verwendet werden. Schreibt in `foo.abc` dieses Beispiel für eine Integer-Variable:

```
@ <stdio.h>

fn main()
{
    local a: readonly int;

    a = 42;
    printf("a = %d\n", a);
}
```

Hier wird vereinbart, dass die Variable `a` im Folgenden nicht mehr verändert werden darf. Übersetzt das Programm, um eine entsprechende Fehlermeldung zu bekommen.

Man sieht hier, dass es wenig Sinn macht, eine lokale `readonly` Variable bei der Deklaration nicht zu initialisieren. Man kann den Wert anschließend nicht setzen, und da es sich um eine lokale Variable handelt, hat diese stets einen undefinierten Wert.

In manchen Fällen soll die Initialisierung einer `readonly` Variable aber vom Wert einer anderen Variablen abhängen. Hier ein Beispiel, bei dem der Benutzer den Wert einer Variable `n` eingeben kann (Zeile 6). Ist der Wert von `n` größer als `10`, soll `a` mit `42` und sonst mit `13` initialisiert werden (Zeile 9 bis Zeile 13):

```

1 @ <stdio.h>
2
3 fn main()
4 {
5     local n: int;
6     scanf("%d", &n);
7
8     local a: readonly int;
9     if (n > 10) {
10         a = 42;
11     } else {
12         a = 13;
13     }
14
15     printf("a = %d\n", a);
16 }
```

Offensichtlich kann dies nicht übersetzt werden, da versucht wird, den Wert von `a` nachträglich zu setzen. Man kann den Wert von `a` aber bei der Initialisierung in Abhängigkeit einer Bedingung wie folgt initialisieren:

```

1 @ <stdio.h>
2
3 fn main()
4 {
5     local n: int;
6     scanf("%d", &n);
7
8     local a: readonly int = n > 10 ? 42 : 13;
9
10     printf("a = %d\n", a);
11 }
```

In Zeile 8 wird hier auf der rechten Seite des Gleichheitszeichens eine sogenannte *Conditional-Expression* verwendet. Ist die Bedingung `n > 10` erfüllt, dann ist der Wert der Expression hier `42` (und allgemein das, was nach dem Fragezeichen kommt) und sonst `13` (und allgemein das, was nach dem Doppelpunkt kommt). Testet das Programm.

In ABC kann man für Conditional-Expressions eine alternative Notation verwenden, bei der man `?` mit `then` und `:` mit `else` ersetzt. Ändert Zeile 8 in:

```
local a: readonly int = n > 10 then 42 else 13;
```

und testet das Programm erneut.

Fazit: Selbst wenn man Conditional-Expressions nicht mag, spätestens wenn `readonly` (oder `const` in C/C++) ins Spiel kommt braucht man so ein Konstrukt.

Schritt 4: Zurück zur Liste

Aufgabe: Nach diesem Exkurs ändert das Programm für die Liste so, dass alle Funktionen zur Ausgabe der Liste einen Zeiger vom Typ

```
-> readonly Node
```

Node als Parameter erhalten.

Schritt 5: Knoten suchen

Aufgabe: Schreibt eine Funktion `isInList`, mit der man prüfen kann, ob in der Liste ein bestimmter Wert enthalten ist. Die Funktion soll `true` zurückgeben, falls dies der Fall ist, und ansonsten `false`. In der Funktion `main` soll man die Funktion wie folgt verwenden können:

```
printf("5 is %s list\n", isInList(list, 5) ? "in" : "not in");
printf("42 is %s list\n", isInList(list, 42) ? "in" : "not in");
```

Schritt 6: Knoten anhängen auf die unschöne und ineffiziente Art (Don't do that at home!)

Bisher können wir nur Knoten am Anfang der Liste einfügen. Für einfach verkettete Listen ist das auch die effizienteste (und einfachste) Methode, um neue Knoten hinzuzufügen. Aber es ist auch möglich, dass man am Ende der Liste einen Knoten anhängt. Eine Funktion `appendToList` kann man so implementieren, dass diese in der Funktion `main` analog zu `prependToList` verwendet werden kann:

```
83     local list: -> Node = nullptr;
84
85     for (local i: int = 1; i <= 10; ++i) {
86         prependToList(&list, i);
87         appendToList(&list, i + 10);
88     }
```

Die Implementierung ist aber im Vergleich zu `prependToList` deutlich komplizierter. Möglich wäre zum Beispiel diese Implementierung:

```
22 fn appendToList(list: ->-> Node, val: int)
23 {
24     local n: -> Node = malloc(sizeof(*n));
25     if (!n) {
26         printf("out of memory\n");
27         exit(1);
28     }
29     n->val = val;
30     n->next = nullptr;
31     if (*list) {
32         for (; (*list)->next; list = &(*list)->next) {
33             ;
34         }
35         (*list)->next = n;
36     } else {
37         *list = n;
38     }
```

In den Zeilen 24 bis 28 wird wie bisher Speicher für den Knoten allokiert. Der Wert des Knotens wird in Zeile 29 gesetzt, und in Zeile 30 wird festgelegt, dass der Knoten keine Nachfolger hat (da dieser ans Ende angefügt wird). Was die Sache nun kompliziert macht, ist, dass man unterscheiden muss, ob man den neuen Knoten an eine nicht-leere Liste anhängt oder in eine leere Liste einfügt. Bei einer leeren Liste ist `*list` der Null-Zeiger und die Bedingung in Zeile 31 `false`.

1. In den Zeilen 32 bis 34 wird der Knoten an das Ende einer nicht-leeren Liste angehängt. Dazu wird mit einer For-Schleife ans Ende der Liste iteriert. Dass hier `list` ein Zeiger auf einen Zeiger auf einen Knoten ist, macht die Sache dabei nicht einfacher. Nach der For-Schleife ist aber `*list` der Zeiger auf den letzten Knoten und an diesen wird der neue Knoten angehängt.
2. In Zeile 36 wird der Fall behandelt, dass die Liste leer ist. In dem Fall wird der Knoten wie in `prependToList` eingefügt.

Um das Anhängen eines Knotens in eine Liste zu unterstützen, lohnt es sich, die Datenstruktur anzupassen, um die Implementierung zu vereinfachen. Aber nicht nur das, bei obiger Implementierung hängt die Laufzeit davon ab, wie lange die Liste ist. Je länger die Liste ist, umso länger benötigt die Operation. Also da gibt's noch viel zu verbessern!!

Nichtsdestotrotz, implementiert `appendToList`, um das grundsätzliche Verständnis für Zeiger und Listen zu üben. Verwendet die Datenstruktur aber in dieser Form nicht im Alltag.

Schritt 7: Praktische Anwendung – eine einfache Symboltabelle

Listen sind in ihrer reinen Form so ziemlich die ineffizienteste Datenstruktur: Wenn man ein Element sucht, muss man im schlechtesten Fall die komplette Liste durchlaufen. Die Zugriffszeit hängt also von der Länge der Liste ab. Dafür sind Listen sehr flexibel: Man muss vorher nicht wissen, wie viele Elemente am Ende gespeichert werden.

Arrays sind bei den Zugriffszeiten viel effizienter. Arrays sind bei den Zugriffszeiten oft schwer zu schlagen. Der Softwareentwickler Scott Meyers brachte dies einmal mit der Bemerkung

“I don't know [data structure], but I know an array will beat it.”

auf den Punkt. Die Aussage ist weniger übertrieben, als sie zunächst klingt: Für viele praktische Probleme sind Arrays aufgrund ihrer einfachen Speicherorganisation und der guten Cache-Ausnutzung tatsächlich schneller als deutlich kompliziertere Datenstrukturen. Mehr dazu in HPC1.

Wir werden uns auf diesem Blatt nicht mit Datenstrukturen wie Hash-Maps beschäftigen, die hier einen guten Kompromiss bieten würden. Stattdessen betrachten wir eine einfache, aber für HPC0-Plus nützliche Anwendung: *eine Symboltabelle*.

Eine Symboltabelle ist eine Datenstruktur, mit der ein Compiler Informationen über Bezeichner verwaltet. In unserem Fall wollen wir globale und lokale Variablen speichern. Dabei sollen folgende Operationen unterstützt werden:

- Mit `symtabAddGlobal((String)"foo")` wird versucht, eine neue globale Variable mit dem Bezeichner `foo` in die Symboltabelle aufzunehmen. Falls es noch keine globale Variable mit diesem Bezeichner gibt, soll das gelingen und die Funktion `true` zurückgeben. Gibt es bereits eine globale Variable mit diesem Namen, soll die Funktion `false` zurückgeben.
- Mit `symtabAddLocal((String)"foo", 8)` wird versucht, eine neue lokale Variable mit dem Bezeichner `foo` und dem Offset `8` aufzunehmen. Falls es noch keine lokale Variable mit diesem Bezeichner gibt, soll das gelingen und die Funktion `true` zurückgeben. Gibt es bereits eine lokale Variable mit diesem Namen, soll die Funktion `false` zurückgeben. Ob es eine globale Variable mit diesem Namen gibt, spielt dabei keine Rolle.
- Mit `symtabPrintGlobals()` und `symtabPrintLocals()` sollen alle globalen beziehungsweise lokalen Variablen sinnvoll ausgegeben werden. Bei den globalen Variablen bedeutet „sinnvoll“, dass Assembler-Code für das BSS-Segment ausgegeben wird.
- Mit `symtabFreeLocals()` sollen alle lokalen Variablen vergessen werden. Diese Funktion braucht man, wenn der Compiler mit der Codegenerierung für eine neue Funktion beginnt.

Hier das in `symtab.hdr` formal beschriebene Interface:

```
1 @ "string.hdr"
2
3 extern fn symtabAddGlobal(ident: String): bool;
4 extern fn symtabAddLocal(ident: String, offset: int): bool;
5 extern fn symtabFreeLocals();
6
7 extern fn symtabPrintGlobals();
8 extern fn symtabPrintLocals();
```

Eingebunden wird dabei die Datei `string.hdr`, die lediglich folgende Zeile enthält:

```
1 type String: array[42] of char;
```

Eine Variable vom Typ `String` ist demnach lediglich ein Array aus genau 42 Zeichen einschließlich des abschließenden Null-Bytes.

Kurz gesagt: Strings sind in C-ähnlichen Sprachen ein erstaunlich komplexes Thema.

Etwas länger gesagt: Dies ist bestimmt nicht die beste Möglichkeit, Strings darzustellen. Wie man „besser“ mit Strings umgeht, ist ein eigenes Thema für eine spätere Session. Denn was „besser“ bedeutet, hängt davon ab, welche Ziele man verfolgt: möglichst einfache Implementierung, geringer Speicherverbrauch, schnelle Zugriffe, flexible Länge oder sichere Speicherverwaltung.

In C, C++ und damit auch in ABC wird keine dieser Zielsetzungen fest vorgegeben. Deshalb gibt es hier nicht die eine richtige Lösung, sondern verschiedene Designentscheidungen mit unterschiedlichen Vor- und Nachteilen.

Der Fokus liegt nun darauf, die Symboltabelle zu implementieren. Bevor wir damit anfangen, brauchen wir aber erst einmal einen kleinen Test. Was bisher in `xtest_syntab.abc` steht, greift noch zu kurz:

```
1 @ <stdio.h>
2
3 @ "string.h"
4 @ "syntab.h"
5
6 fn main()
7 {
8     // add some global and local variables
9     assert(syntabAddGlobal((String)"foo"));
10    assert(!syntabAddGlobal((String)"foo"));
11    syntabAddLocal((String)"bar", -8);
12    printf("GLOBAL VARIABLES:\n");
13    syntabPrintGlobals();
14    printf("LOCAL VARIABLES:\n");
15    syntabPrintLocals();
16 }
```

Aufgabe:

Schreibt `syntab.h`, `string.h` und `xtest_syntab.abc` ab. Eine ausführbare Datei könnt ihr noch nicht erzeugen, da `syntab.abc` noch fehlt. Mit

```
abc -c xtest_syntab.abc
```

könnt ihr aber prüfen, ob sich ein Objectfile erzeugen lässt, ob also das Interface schon passt und richtig verwendet wird.

Erweitert anschließend das Testprogramm so, dass mindestens folgende Fälle getestet werden:

- Eine weitere globale Variable wird erfolgreich eingefügt.
- Dieselbe globale Variable wird erneut eingefügt. Das muss fehlschlagen.
- Eine lokale Variable wird erfolgreich eingefügt.
- Dieselbe lokale Variable wird erneut eingefügt. Das muss fehlschlagen.
- Eine lokale Variable darf denselben Namen wie eine globale Variable haben.
- Nach `syntabFreeLocals()` darf derselbe lokale Name erneut verwendet werden.
- Ruft an geeigneten Stellen `syntabPrintGlobals()` und `syntabPrintLocals()` auf, um zu testen, dass es die Variablen auch wirklich gibt.

Hinweis:

Zum Testen der Symboltabelle kann man `assert` verwenden. Wenn das Einfügen einer Variablen erfolgreich sein soll, schreibt man beispielsweise:

```
assert(syntabAddGlobal((String)"foo"));
```

Wenn das Einfügen fehlschlagen soll, weil bereits eine Variable mit diesem Namen existiert, schreibt man:

```
assert(!syntabAddGlobal((String)"foo"));
```

Schritt 8: Implementierung der Symboltabelle

Als Nächstes legen wir die Datei `syntab.abc` an. Zunächst enthält sie nur Stubs für die Funktionen, die wir anschließend implementieren wollen:

```
1 @ <stdlib.hdr>
2 @ <stdio.hdr>
3 @ <string.hdr>
4
5 @ "syntab.hdr"
6
7 struct Node {
8     ident: String;
9     offset: int;
10    next: -> Node;
11 };
12
13 global globalVar, localVar: -> Node;
14
15 fn isInList(list: -> readonly Node, ident: String): bool
16 {
17     // TODO
18     return false;
19 }
20
21 fn syntabAddGlobal(ident: String): bool
22 {
23     return false;
24 }
25
26 fn syntabAddLocal(ident: String, offset: int): bool
27 {
28     return false;
29 }
30
31 fn syntabFreeLocals()
32 {
33     // TODO
34 }
35
36 fn syntabPrintGlobals()
37 {
38     // TODO
39 }
40
41 fn syntabPrintLocals()
42 {
43     // TODO
44 }
```

In den folgenden Aufgaben werden diese Funktionen Schritt für Schritt implementiert. Dabei kann man das bisher Gelernte über einfach verkettete Listen vertiefen. Ein paar Kleinigkeiten sind aber anders:

- Wir haben jetzt zwei Listen: `globalVar` und `localVar`.
- Die Listen sind nur innerhalb des Moduls `syntab.abc` sichtbar. In `list.abc` hatte der Benutzer selbst einen Zeiger auf die Liste.

Das hat zwei Vorteile: Erstens kann man die tatsächlich verwendete Datenstruktur später austauschen, ohne das Interface ändern zu müssen. Zweitens müsst ihr deshalb an einigen Stellen etwas mitdenken ;-)

Aufgabe

1. Mit `abc xtest_symtab.abc symtab.abc -o xtest_symtab` könnt ihr jetzt bereits das Testprogramm erzeugen. Mit `./xtest_symtab` könnt ihr es ausführen. Das Programm sollte natürlich noch mit einem Assertion-Failure abbrechen.
2. Fangt mit `symtabAddGlobal` und `symtabAddLocal` an. In einer ersten Version soll stets ein neuer Knoten am Anfang der jeweiligen Liste eingefügt werden, egal ob bereits ein Eintrag mit diesem `ident` in der Liste vorhanden ist. Die Funktionen sollen also zunächst immer erfolgreich sein und `true` zurückgeben.

Im Testprogramm könnt ihr vorübergehend die Tests auskommentieren, die erwarten, dass das Einfügen fehlschlägt.

3. Implementiert anschließend `symtabPrintGlobals` und `symtabPrintLocals`, sodass ihr überprüfen könnt, ob die Knoten wirklich in die richtige Liste eingefügt wurden.

Enthält die Liste `globalVar` beispielsweise Knoten für die Bezeichner `x` und `y`, dann soll `symtabPrintGlobals()` folgende Ausgabe erzeugen:

```
.bss
.align 8
x:
    .quad
    .align 8
y:
    .quad
```

Die Ausgabe orientiert sich dabei an dem Assembler-Code, den ein Compiler für globale, nicht-initialisierte Variablen erzeugt.

Enthält die Liste `localVar` beispielsweise Knoten für die Bezeichner `x` und `y` mit den Offsets `-8` und `24`, dann soll `symtabPrintLocals()` folgende Ausgabe erzeugen:

```
x with offset -8
y with offset 24
```

Hinweis: Da neue Knoten am Anfang der Liste eingefügt werden, erscheint in beiden Fällen die Ausgabe möglicherweise in umgekehrter Reihenfolge der Einfügungen. Aber die Reihenfolge ist egal.

4. Implementiert nun die Funktion `isInList`. Die Verwendung dieser Funktion ist denkbar einfach. Zu Beginn von `symtabAddGlobal` und `symtabAddLocal` prüft man, ob der Bezeichner bereits in der entsprechenden Liste vorhanden ist. In `symtabAddGlobal` zum Beispiel mit

```
if (isInList(globalVar, ident)) {
    return false;
}
```

Ist der Bezeichner bereits vorhanden, wird die Funktion sofort mit `false` beendet. Andernfalls wird wie bisher ein neuer Knoten angelegt und eingefügt.

Die Implementierung von `isInList` ist fast identisch mit der Funktion aus Schritt 5. Es gibt allerdings eine wichtige Feinheit zu beachten:

Strings können nicht mit dem Operator `==` auf Gleichheit überprüft werden. Der Hintergrund ist, dass in ABC (und in C) der Operator `==` genau dann verwendet werden kann, wenn beide Operanden in Register geladen, verglichen und anschließend das Zero-Statusflag ausgewertet werden kann. Ein String besteht jedoch aus vielen Zeichen und passt nicht in ein einzelnes Register.

In `<string.hdr>` (also nicht unserer eigenen Header-Datei `string.hdr`, sondern der Header-Datei aus der Standardbibliothek) ist die Funktion `strcmp` deklariert. Mit ihr kann man zwei Strings vergleichen: `strcmp(s1, s2)` liefert den Wert 0, wenn die Strings gleich sind, einen negativen Wert, wenn `s1` lexikografisch vor `s2` kommt und einen positiven Wert, wenn `s1` lexikografisch nach `s2` kommt.

Für die Gleichheitsprüfung interessiert uns daher nur der Fall `strcmp(s1, s2) == 0`

Bemerkung: Bei uns sind `s1` und `s2` vom Typ `String` (also Arrays mit 42 Zeichen). Bei `strcmp(s1, s2)` wandelt der ABC-Compiler dann den Aufruf implizit in `strcmp(&s1[0], &s2[0])` um. Übergeben wird also jeweils ein Zeiger auf das erste Zeichen des Strings.